Riemannian Optimization Techniques and Applications

Honors Thesis

Sai Nikhil Chanda

A honors thesis presented for the degree of
Bachelors of Arts with Honors



Department of Mathematics
Clark University
United States of America
April 22, 2026

Abstract

This paper explores numerical methods to solving optimization problems on Riemannian manifolds. Gradient Descent is a popular tool to numerically calculate extrema when no closed form solution exists. However, traditional Gradient Descent fails when data lies on curved or constrained spaces that cannot be expressed as Euclidean space. To remedy this, we discuss techniques to extend Gradient Descent to Riemannian manifolds.

In this paper, we provide a summary of necessary preliminary information including the motivation, mathematical understanding, and proof for all subsequent methods. Those who require a more basic understanding of differential geometry, refer to [1]. Most of the concepts are taken from Nicolas Boumal's *An Introduction to Optimization on Smooth Manifolds*, however there is material supplemented from other sources as well. The appendix will provide algorithms, computation, as well as longer proofs.

Next, we will dive into the Frank-Wolfe algorithm, first designed in 1956. We will go over the motivation, the iteration, convergence, and applications. Then we will discuss the extension of the FW algorithm to the Riemannian setting, allowing us to solve problems on manifold settings.

Contents

1 Preliminaries **2**
- 1.1 Riemannian Gradients . . . 2
- 1.2 Orthogonal Projection . . . 3
- 1.3 Optimization and Convexity . . . 4

2 Gradient Descent **5**
- 2.1 Riemannian Gradient Descent Algorithm . . . 5
- 2.2 First Order Optimality . . . 6
- 2.3 Convergence and Complexity Conditions . . . 7
- 2.4 Backtracking Method . . . 10
- 2.5 Newton Method . . . 12

3 Constrained Convex Optimization Techniques **13**
- 3.1 Preliminary Methods . . . 13
  - 3.1.1 Projected Gradient Descent . . . 13
  - 3.1.2 Interior-Point Method . . . 13
- 3.2 Frank-Wolfe Algorithm . . . 14
  - 3.2.1 Convergence and Complexity . . . 15

4 Application **18**
- 4.1 Principal Component Analysis (PCA) . . . 18
  - 4.1.1 PCA Computation . . . 19
  - 4.1.2 Simulation . . . 21
- 4.2 Geometric Matrix Mean . . . 21
  - 4.2.1 Computations on HPD matrices . . . 22
  - 4.2.2 Simulation . . . 24

5 Appendix **27**
- 5.1 PCA Simulation Code . . . 27
- 5.2 HPD Simulation Code . . . 28

1 Preliminaries

This section will cover topics necessary to understand the techniques used later on. Lets first do a quick review of Riemannian geometry. However, a more in depth understanding can be found at [14]

Let $\mathcal{E}$ be a linear space of dimension d. An *embedded submanifold* $\mathcal{M}$ is a non-empty subset of $\mathcal{E}$ of dimension $n = d-k$ for $k \geq 1$ and for each $x \in \mathcal{M}$, $\exists$ a neighborhood $\mathcal{U}_x \subset \mathcal{E}$, $V \subset \mathbb{R}^d$, and a diffeomorphism $F : \mathcal{U} \to V$ such that $F(\mathcal{M} \cap \mathcal{U}) = E \cap V$ where $E = \{y \in \mathbb{R}^d : y_{n+1}, \cdots, y_d = 0\}$. Locally, these spaces are Euclidean and are endowed with a differential structure. If we let $\mathcal{M}$ and $\mathcal{M}'$ be embedded submanifolds of $\mathcal{E}$ and $\mathcal{E}'$, we can say the map $F : \mathcal{M} \to \mathcal{M}'$ if $\exists \text{asmoothmap} \bar{F} : \mathcal{U} \in \mathcal{E} \to \mathcal{E}' \text{s.t.} F(y) = \bar{F}(y)|_{\mathcal{M}\cap\mathcal{U}}$. We say that $\bar{F}$ is a local smooth extension of F around x.

At any point $x \in \mathcal{M}$, we define a linear space $T_x\mathcal{M} \equiv \mathbb{R}^n$ that contain all the vectors tangent to x. Combined with the *Riemannian metric*, which defines an inner product $\langle \cdot, \cdot \rangle_x$ smoothly with x, we define a *Riemannian manifold* as a manifold with a Riemannian metric.

The norm of any vector $v \in T_x\mathcal{M}$ is defined as $\|v\|_x = \sqrt{\langle v, v \rangle_x}$. The tangent bundle $T\mathcal{M} = \{(x, v) \in \mathcal{M} \times T_x\mathcal{M}\} \equiv \mathbb{R}^{2n}$ of a manifold $\mathcal{M}$ is the disjoint union of $\mathcal{M}$ and $T_x\mathcal{M}$. A *retraction* $R : TM \to \mathcal{M}$ is a smooth map such that each curve $c(t) = R_x(tv)$ with $c(0) = x$ and $c'(0) = v$. Note that a curve $c(t) : I \subset \mathbb{R} \to \mathcal{M}$ is a smooth map. We can also say the same thing for a scalar function $f : \mathcal{M} \to \mathbb{R}$ and $\bar{f} : \mathcal{U} \to \mathbb{R}$.

A special type of retraction is the *Exponential map* $\text{Exp}_x : T_x\mathcal{M} \to \mathcal{M}$ such that $y = \text{Exp}_x(v) \in \mathcal{M}$ along a geodesic $\gamma : [0, 1] \to \mathcal{M}$ such that $\gamma(0) = x, \gamma'(0) = v$. Because the exponential map is a diffeomorphism, we can define the inverse as $\text{Exp}_x^{-1} : \mathcal{M} \to T_x\mathcal{M}$. An issue that arises in differential geometry is when we want to compare tangent vectors v, u where $v \in T_x\mathcal{M}$ and $u \in T_y\mathcal{M}$ when $x \neq y$. To solve this, we compute the parallel transport, where we move v along the geodesic γ where $\gamma(0) = x, \gamma(1) = y$ to $T_y\mathcal{M}$. This transported vector is denoted by $\Gamma_x^y v$.

1.1 Riemannian Gradients

Definition 1. *Let $f : \mathcal{M} \to \mathbb{R}$ be smooth on a Riemannian manifold $\mathcal{M}$. The Riemannian gradient of f is a vector field* $\text{grad} f$ *on $\mathcal{M}$ uniquely defined by:*

$$\forall (x, v) \in T\mathcal{M} \qquad Df(x)[v] = \langle v, \text{grad} f \rangle_x \tag{1}$$

where $Df(x)[v]$ is $\frac{d}{dt} f(c(t))|_{t=0}$, $c(t)$ is a smooth curve, and $\langle \cdot, \cdot \rangle_x$ is the Riemannian metric.

Proposition 1. *Let $f : \mathcal{M} \to \mathbb{R}_x$ be smooth on $\mathcal{M}$ with retraction R. Then $\forall x \in \mathcal{M}$*

$$\text{grad} f(x) = \text{grad}(f \circ R_x)(0)$$

where $f \circ R_x : T_x\mathcal{M} \to \mathbb{R}$ is defined on Euclidean space, meaning the gradient is the classical gradient

Proof. By the chain rule, $\forall v \in T_x\mathcal{M}$,

$$D(f \circ R_x)(0)[v] = Df(R_x(0))[R_x(0)[v]] = Df(x)[v]$$

the second inequality comes from the fact that $R_x(0)$ is an identity mapping. Because of this equality, we can say that $Df(x)[v] = \langle \text{grad}(f \circ R_x)(0), v \rangle_x$, meaning the gradient must be unique.

□

If $\mathcal{M}$ is an embedded manifold on Euclidean space $\mathcal{E}$ that has a metric $\langle \cdot, \cdot \rangle$. Since f is smooth, it has a smooth extension $\bar{f}$ defined on a neighborhood of $\mathcal{M}$ in $\mathcal{E}$. We say that $\bar{f}$ has the Euclidean gradient $\text{grad} \bar{f}$.

$$\begin{aligned} \forall (x, v) \in T\mathcal{M} \qquad \langle v, \text{grad} f(x) \rangle_x &= Df(x)[v] \\ &= D\bar{f}(x)[v] \\ &= \langle v, \text{grad} f(x) \rangle \end{aligned}$$

Computing derivatives can be done by first computing the directional derivative $Df(x)[v]$ through the definition, and then turning it into the form of $\langle v, \cdot \rangle_x$. Usually to compute directional derivative, we consider the smooth extension. Before we go into an example, lets define the notion of projection onto a manifold.

1.2 Orthogonal Projection

Definition 2. *Let $\mathcal{M}$ be an embedded submanifold on Euclidean space $\mathcal{E}$ with a metric $\langle \cdot, \cdot \rangle$. The orthogonal projection to $T_x\mathcal{M}$ is the linear map* $\mathrm{Proj}_x : \mathcal{E} \to \mathcal{E}$

1. *Range: Im*$(\mathrm{Proj}_x) = T_x\mathcal{M}$
2. *Projector:* $\mathrm{Proj}_x \circ \mathrm{Proj}_x = \mathrm{Proj}_x$
3. *Orthogonal:* $\langle u - \mathrm{Proj}_x, v \rangle = 0 \quad \forall v \in T_x\mathcal{M}$ *and* $u \in \mathcal{E}$

If $\mathcal{M}$ is an open submanifold, then Proj_x *is the identity map because $T_x\mathcal{M} \equiv \mathcal{E}$*

Example 1. *Consider the manifold S^{d-1}, which is unit sphere equipped with the metric $\langle u, v \rangle_x = u^\top v$. Also consider the smooth function $f : S^{d-1} \to \mathbb{R}$ where $f(x) = x^\top Ax$. For the simplicity of computation, let $A \in \mathrm{Sym}_d(\mathbb{R})$, meaning that $A^\top = A$. Then the directional derivative is*

$$D\bar{f}(x)[v] = 2x^\top Av = \langle 2Ax, v \rangle.$$

Then $\mathrm{grad}\bar{f} = 2Ax$. *To find* $\mathrm{grad}f$, *we need to project it back to $T_xS^{d-1} = \{v \in \mathbb{R}^d : \langle v, x \rangle = 0\}$ using the fact* $\mathrm{Proj}_x(u) = u - (x^\top u)x = (I_d - xx^\top)u$

$$\begin{aligned} \mathrm{grad}f(x) = \mathrm{Proj}_x(\mathrm{grad}\bar{f}(x)) &= (I_d - xx^\top)(2Ax) && (2)\\ &= 2Ax - x^\top 2Axx && (3)\\ &= 2Ax - 2x^\top Axx && (4)\\ &= 2(Ax - (x^\top Ax)x) && (5) \end{aligned}$$

When considering the case $\mathrm{grad}f(x) = 0$, *this implies that* $\mathrm{Proj}_x(0) = 0$ *and using (5) gives*

$$Ax = (x^\top Ax)x = \lambda x \quad \text{for } \lambda \in \mathbb{R}, \tag{6}$$

implying the gradient vanishes at the eigenvectors $x \in \mathcal{E}$.

Another important result of the orthogonal projector is that it is self adjoint. Recall the adjoint of an operator A is $A^\dagger$ where $\langle Ax, u \rangle = \langle x, A^\dagger u \rangle$. Adjoints come up in many applications of optimization and control theory, as it allows us to calculate gradients in numerical settings.

Proposition 2. *Let* Proj_x *be the orthogonal projector from $\mathcal{E}$ to a linear subspace of $\mathcal{E}$. Then* Proj_x *is self-adjoint, meaning that* $\langle \mathrm{Proj}_x(v)x, u \rangle = \langle v, \mathrm{Proj}_x(u) \rangle$

Proof. From the properties of orthogonal projectors, $\forall u, v \in \mathcal{E}$

$$\begin{aligned} 0 &= \langle u - \mathrm{Proj}_x(u), \mathrm{Proj}_x(v) \rangle \\ &= \langle u, \mathrm{Proj}_x(v) \rangle - \langle \mathrm{Proj}_x(u), \mathrm{Proj}_x(v) \rangle \\ &= \langle u, \mathrm{Proj}_x(v) \rangle - \langle \mathrm{Proj}_x(u), v - (v - \mathrm{Proj}_x(v)) \rangle \\ &= \langle u, \mathrm{Proj}_x(v) \rangle - \langle \mathrm{Proj}_x(u), v \rangle - \langle \mathrm{Proj}_x(u), v - \mathrm{Proj}_x(v) \rangle \\ &= \langle u, \mathrm{Proj}_x(v) \rangle - \langle \mathrm{Proj}_x(u), v \rangle \end{aligned}$$

concluding the proof. □

1.3 Optimization and Convexity

An important part to understanding optimization is knowing the constraints in regards to a problem. We call this a constrained optimization problem, which is of the form

$$\min_{x \in S} f(x). \tag{7}$$

Here, we call S the feasible region and f the objective function. Note the problem can also be a max problem and would be written the same. For our purposes of this paper, we will only concern ourselves with convex feasible regions. Some simple examples of convex feasible regions are polyhedrons, spheres, or the set of positive definite matrices. Its important to bolster the meaning of what it means for a set to be convex.

Definition 3. *A set S is convex if for all $x, y \in S$*

$$\lambda x + (1-\lambda)y \in S, \qquad \lambda \in (0,1) \tag{8}$$

and when S is a subset of a Riemannian manifold $\mathcal{M}$, it is considered to be geodesically convex if $\forall x, y \in S, \exists$ a geodesic segment $c : [0,1] \to \mathcal{M}$ such that $c(0) = x$, $c(1) = y$, and $c(t) \in S \forall\ t \in [0,1]$.

In particular, no matter the setting, the convex combination must stay inside the set for that set to be convex. Note we only consider λ in the open set of $(0,1)$ because $\lambda = 0, 1$ cases are trivial. Similarly, we can define the notion of convexity for an objective function.

Definition 4. *Let $f : S \to \mathbb{R}$ be a differentiable function. Then f is a convex function if*

$$f(y) - f(x) \geq \langle \text{grad} f(x), y - x \rangle \qquad \forall\ x, y \in S, \tag{9}$$

and it is μ-strongly convex if

$$f(y) - f(x) \geq \langle \text{grad} f(x), y - x \rangle + \frac{\mu}{2}\|y - x\|^2 \qquad \forall\ x, y \in S. \tag{10}$$

To prove convergence for many of our optimization methods, its important to define a notion of smoothness. For the rest of the definitions, we will consider then Riemannian manifold space.

Definition 5. *A map $f : \mathcal{M} \to \mathbb{R}$ is L-smooth, or that it has L-Lipschitz gradients, if*

$$\|\text{grad} f(x) - \Gamma_x^y \text{grad} f(x)\| \leq L d(x, y) \qquad \forall\ x, y \in \mathcal{M}, \tag{11}$$

where $d(x, y)$ represents the geodesic distance between x and y. Equivalently, we can say that

$$f(y) \leq f(x) + \left\langle v, \text{Exp}_x^{-1}(y) \right\rangle_x + \frac{L}{2} d^2(x, y) \qquad \forall x, y \in \mathcal{M}; v \in T_x\mathcal{M} \tag{12}$$

Definition 6. *We say that a differentiable function $f : \mathcal{M} \to \mathbb{R}$ is geodesically convex (g-convex) if*

$$f(y) \geq f(x) + \left\langle v, \text{Exp}_x^{-1}(y) \right\rangle_x \qquad \forall x, y \in \mathcal{M}; v \in T_x\mathcal{M} \tag{13}$$

and we call it μ-strongly g-convex if

$$f(y) \geq f(x) + \left\langle v, \text{Exp}_x^{-1}(y) \right\rangle_x + \frac{\mu}{2} d^2(x, y). \qquad \forall x, y \in \mathcal{M}; v \in T_x\mathcal{M}; \mu \geq 0 \tag{14}$$

An important result of these definitions it to show that local optimizers of our constrained problem are globally optimal.

Proposition 3. *Let $x^* \in S \subset \mathcal{M}$ be a local optimum for (7). Then, x^* is globally optimal, and*

$$\left\langle \text{grad} f(x^*), \text{Exp}_{x^*}^{-1}(y) \right\rangle_{x^*} \geq 0 \tag{15}$$

for all $y \in S$.

Proof. Assume $x^* \in S \subset \mathcal{M}$ is a local optimizer that is not a global minimizer. Then there exists $y \in S$ such that $f(y) < f(x)$. There also exists a geodesic connecting $c(0) = x$ to $c(1) = y$ in S such that for all $t \in [0, 1]$,

$$f(c(t) \leq (1-t)f(x) - tf(y) = f(x) + t(f(y) - f(x)) < f(x), \tag{16}$$

which contradicts the local optimality. To show the inequality, we need to differentiate $f(c(t))$ at $t = 0$. An important fact is that because x^* is a local optimum, the derivative must be positive.

$$0 \leq \frac{d}{dt}\bigg|_{t=0} f(c(t)) = \langle \text{grad} f(x^*), c'(0) \rangle = \langle \text{grad} f(x^*), \text{Exp}_{x^*}^{-1}(y) \rangle_{x^*} \tag{17}$$

□

2 Gradient Descent

In optimization, we are generally given a problem where $f : \mathbb{R}^d \to \mathbb{R}$ is a smooth function and we want to find x^* such that this inequality is satisfied.

$$f(x^*) \leq \min_{x \in \mathbb{R}^d} f(x) + \epsilon \tag{18}$$

One method to solving this is by solving a sub problem, where we look for points such that the gradient is near 0. We will go more in-depth on what it means for a point to have a 0 gradient, however note that this method produces candidates i.e. critical points. This method is known as the Gradient Descent method. Gradient Descent is a first-order optimization algorithm that looks to find the root of grad$f$ numerically through an iterative step process. The equation

$$x_{k+1} = x_k - \alpha_k \text{grad} f(x_k) \qquad \alpha_k \in \mathbb{R} \tag{19}$$

produces a convergent sequence $\{x_k\}_{k=0}^{\infty} \to p$ such that grad$f(p) \approx 0$. Gradient descent offers an approach to calculate the local minimum or maximum of a function when there is no closed form solution. In the sense of Riemannian manifolds, we can express this

$$x_{k+1} = R_{x_k}(-\alpha_k \text{grad} f(x_k)) \qquad \alpha_k \in \mathbb{R} \tag{20}$$

where R_{x_k} is a retraction. We will later talk about how we can optimize our step sizes α_k through another algorithm. However, there are three main options as to how to define α_k, being:

1. Fixed step-sizes: $\alpha_k = \alpha \ \forall \ k$
2. Optimal step-size α_k minimizes $g(t)$ exactly; however, can only be done cheaply in a few cases
3. Backtracking: starting with a guess $t_0 > 0$, iteratively reduce it by a factor as $t_i = \tau t_{i-1}$ with $\tau \in (0, 1)$ until t_i is deemed acceptable, and set $\alpha_k = t_i$. We will show later the various ways to pick t_0.

2.1 Riemannian Gradient Descent Algorithm

It is clear to see that if we let $\mathcal{M}$ be an open submanifold of $\mathcal{E}$, then the gradient descent step is same as equation (12). However, we wish to find a solution for $\min_{x \in \mathcal{M}} f(x)$, we need to consider the iteration steps

$$x_{k+1} = g(x) = f(R_{x_k}(s_k)) \tag{21}$$

We can approximate $g(x)$ with a first-order Taylor series expansion. If we let $R_x(tv) = c(t)$ where $c : I = (-\epsilon, \epsilon) \to M$ with the initial conditions $c(0) = x, c'(0) = v$. If we let $g(t) = f(c(t))$ where $f : \mathcal{M} \to \mathbb{R}$ where f is a cost function, the first-order Taylor series expansion is:

$$g(t) = g(0) + tg'(t) + O(t^2); \quad g(0) = f(x) \tag{22}$$

Using chain rule, we can show $g'(t) = Df(c(t))[c'(t)] = \langle \text{grad} f(c(t), c'(t) \rangle_x + O(t^2)$. This means that $g'(0) = \langle \text{grad} f(x), v \rangle_x$. If we now say that $c(t) = R_x(tv)$ where R_x is a retraction, then

$$g(s) = f(R_x(s)) = f(x) + \langle \text{grad} f, s \rangle_x + O(\|s\|_x^2) \tag{23}$$

Now we can outline the algorithm for Riemannian Gradient Descent (RGD).

Algorithm 1 RGD: Riemannian Gradient Descent method

Require: Initial point $x_0 \in \mathcal{M}$
1: **for** $k = 0, 1, 2, \ldots$ **do**
2: Pick a step-size $\alpha_k > 0$
3: $x_{k+1} = R_{x_k}(s_k)$, with step $s_k = -\alpha_k \text{grad} f(x_k)$
4: **end for**
Ensure: Sequence (x_k)

Before we can show Gradient descent produces a convergent sequence to a root, we first need more mechanics on the topology of sequences.

2.2 First Order Optimality

To ensure that this generalized RGD method generates a converging sequence, its important to analyze the output and conditions necessary to ensure convergence to a critical point. To do this, we cover some definitions from topology and what it means to be a critical point on a Riemannian manifold.

Definition 7. *Consider a sequence $S = x_0, x_1, x_2, \cdots$ on a manifold $\mathcal{M}$. Then*

1. *$x \in \mathcal{M}$ is a limit of S if, for every $\mathcal{U}_x \subseteq \mathcal{M}$, $\exists$ K such that $x_k, x_{k+1}, x_{k+2}, \cdots \in \mathcal{U}_x$. The topology of a manifold is Hausdoff, hence a sequence has at most one limit. If x is a limit, we write $\lim_{k\to\infty} x_k = x$ or $x_k \to x$ and say the sequence converges to x.*
2. *$x \in \mathcal{M}$ is an accumulation point of S if it is the limit point of a subsequence of S, that is, if every neighborhood $\mathcal{U}_x \subseteq \mathcal{M}$ contains an infinite number of elements in S.*

Example 2. 1. *$x_k = (-1)^k$ has no limit points but has 2 accumulation points at $x = \pm 1$*

2. $x_k = \begin{cases} k & k \in 2\mathbb{Z}, \\ 1/k & \text{otherwise} \end{cases}$

   has a single accumulation point at $x = 0$ but no limit point

Definition 8. *$x \in \mathcal{M}$ is critical for a smooth function $f : \mathcal{M} \to \mathbb{R}$ if*

$$(f \circ c)'(0) \geq 0 \to Df(x) = 0 \tag{24}$$

for all smooth $c(t)$ on $\mathcal{M}$ with $c(0) = x$.

This means that when we are at $x \in \mathcal{M}$, f does not initially decrease in any direction. Also note that this applies for minimization problems, as a critical point to a maximization problem must have an initial decrease in all directions.

Proposition 4. *Any local minimizer of a smooth function $f : \mathcal{M} \to \mathbb{R}$ is a critical point of f*

Proof. Let x be a local minimizer of $f : \exists\, \mathcal{U}_x \subseteq \mathcal{M}$ such that $f(x) \leq f(y)\ \forall\ y \in \mathcal{U}_x$. Assume $\exists\ c : I \to \mathcal{M}$ with $c(0) = x$ such that $(f \circ c)'(0) < 0$. Then $\exists\ \delta > 0$ such that

$$(f \circ c)'(\tau) < 0 \quad \forall\ \tau \in [0, \delta] \tag{25}$$

$$f(c(t)) = f(c(0)) + \int_0^t (f \circ c)'(\tau) d\tau < f(x) \quad \forall\, t \in [0, \delta] \tag{26}$$

Since c is continuous $c^{-1}(\mathcal{U}_x) = \{t \in I : c(t) \in \mathcal{U}_x\}$ is open and contains 0 because $c(0) = x \in \mathcal{U}_x$. But this means that $c^{-1}(\mathcal{U}_x) \cap (0, \delta] \neq \emptyset$, so there must exist a $t \in (0, \delta]$ such that $f(c(t)) < f(x)$ where $c(t) \in \mathcal{U}_x$. This is a contradiction because we said that x is a local minimizer. □

It is easy to verify that local maximizers are also critical points. On Riemannian manifolds, its an important fact that the critical points of a function satisfy the vanishing Riemannian gradient.

Proposition 5. *Let $f : \mathcal{M} \to \mathbb{R}$ be a smooth function on a Riemannian manifold $\mathcal{M}$. Then x is a critical point if and only if* $\text{grad} f(x) = 0$.

Proof. Let $c : I \to \mathcal{M}$ be a smooth curve satisfying the initial conditions $c(0) = x, c'(0) = v$. Then from definition,

$$(f \circ c)'(0) = Df(x)[v] = \langle \text{grad} f(x), v \rangle) x \tag{27}$$

If $\text{grad} f(x) = 0$ then x is a critical point since $Df(x)[v] = 0$. If x is a critical point, then

$$\langle \text{grad} f(x), v \rangle_x \geq 0 \qquad \forall\, v \in T_x \mathcal{M} \tag{28}$$

if we consider v and $-v$, then $\text{grad} f(x) = 0$ for all vectors in the tangent space at x. □

2.3 Convergence and Complexity Conditions

Given this notion of a critical point and what it means for a point to be critical on a Riemannian manifold, we can start to discuss the correctness of the RGD algorithm. For our later proofs on complexity and convergence, we need to consider two assumptions on our cost function. Later, we will try to weaken these assumptions.

Assumption 1. f is globally lower-bounded, meaning $\exists\ f_{low} \in \mathbb{R}$ such that $f(x) \geq f_{low}\ \forall\ x \in \mathcal{M}$.

Assumption 2. At each iteration, the algorithm achieves sufficient decrease for f, i.e. $\exists\ c > 0$ such that $\forall\ k f(x_k) - f(x_{k-1}) \geq c \| \text{grad} f(x_k) \|^2$.

An important consequence of the two assumptions is that all accumulation points are critical points, and that as we let the itinerants go to infinity, the gradient vanishes. This is a fundamental result for any algorithm.

Proposition 6. *Let $f : \mathcal{M} \to \mathbb{R}$ be a smooth function on a Riemannian manifold $\mathcal{M}$ with Assumption 1(A1). Let $x_0, x_1, x_2, \cdots$ be iterations with Assumption 2 (A2) with $c > 0$. Then*

$$\lim_{k \to \infty} \| \text{grad} f(x_k) \| = 0 \tag{29}$$

This also implies that all accumulation points are critical points. For all $k \geq 1,\ \exists\ k \in \{0, 1, \cdots, K-1\}$ such that

$$\| \text{grad} f(x_k) \| = \sqrt{\frac{f(x_0) - f_{low}}{c}} \frac{1}{\sqrt{k}} \tag{30}$$

Proof. For all $K \geq 1$,

$$\begin{aligned} f(x_0) - f_{low} &\geq f(x_0) - f(x_K) \\ &= \sum_{k=0}^{K} f(x_k) - f(x_{k+1}) \\ &\geq K c \min_{k=0,1,\ldots,K-1} \| \text{grad} f(x_k) \|^2 \end{aligned}$$

The first inequality comes from the first assumption that there exist that lower bound f_{low}. The second inequality comes from the second assumption and the sense of sufficient decrease. Observe that $f(x_k) \geq f(x_{k+1}) \; \forall \; k$ by A2. Then as we take K to infinity, that gives us:

$$f(x_0) - f_{low} \geq \sum_{k=0}^{\infty} f(x_k) = f(x_{k+1}) \tag{31}$$

The bound implies that the summands must converge to 0, resulting in $\|\text{grad} f(x_k)\| \to 0$. Now to prove the second part, let x be an accumulation point of the sequence of iterations outputted by Algorithm 1. By definition, $\exists$ a subsequence of iterations $x_{(0)}, x_{(1)}, x_{(2)}, ...$ which converge to x. Since $\|\text{grad} f\|$ is continuous, it commutes with the limit.

$$\begin{aligned} 0 &= \lim_{k\to\infty} \|\text{grad} f(x_k)\| \\ &= \lim_{k\to\infty} \|\text{grad} f(x_{(k)})\| \\ &= \|\text{grad} f(\lim_{k\to\infty} x_{(k)})\| \\ &= \|\text{grad} f(x)\| \end{aligned}$$

□

Given our assumptions, its easy to see the iterations outputted by Algorithm 1 converge to a critical point. However, these assumptions are strong, and will need to be relaxed in order to be more applicable. First, understand to ensure Assumption 2, the following inequality must hold:

$$f(x_{k+1}) = f(R_{x_k}(s_k)) = f(x_k) + \langle \text{grad} f(x_k), s_k \rangle + O(\|s_k\|^2) \tag{32}$$

With this equation, we can state our third assumption.

Assumption 3. For a given subset $S \subseteq T\mathcal{M}$, $\exists \; L > 0$ such that for all $(x, s) \in T\mathcal{M}$:

$$f(R_{x_k}(s_k)) \leq f(x_k) + \langle \text{grad} f(x_k), s_k \rangle + \frac{L}{2}\|s_k\|^2 \tag{33}$$

This Lipschitz-type inequality says that there exists a range of step-sizes that lead to a sufficient decrease. We can use this weaker and more applicable assumption to prove Assumption 2 and conjure a value for c. We can also check the assumption's plausibility by referring back to Euclidean space. In a linear space with $x, s \in \mathcal{E}$, the retraction $R_x(s) = x + s$. So if $f \circ R$ satisfies Assumption 3 on $T\mathcal{E}$, then

$$f(x+s) \leq f(x) + \langle \text{grad} f(x), s \rangle + \frac{L}{2}\|s\|^2 \qquad \forall \; (x, s) \in T\mathcal{E} \equiv \mathcal{E} \times \mathcal{E}$$

$$f(x+s) - (f(x) + \langle \text{grad} f(x), s \rangle) \leq \frac{L}{2}\|s\|^2 \tag{34}$$

This inequality expresses that the difference between f and its first-order Taylor expansion is upper-bounded by a quadratic. If the gradient of f is *Lipschitz continuous* with a constant L, meaning:

$$\|\text{grad} f(y) - \text{grad} f(x)\| \leq L\|y - x\| \qquad \forall \; x, y \in \mathcal{E},$$

then equation (34) holds. And if we let $c(t) = x + ts$, then we can use calculus to show that

$$
\begin{aligned}
f(x+s) - f(x) &= f(c(1)) - f(c(0)) && (35)\\
&= \int_0^1 (f \circ c)'(t)dt && (36)\\
&= \int_0^1 Df(c(t))[c'(t)]dt && (37)\\
&= \int_0^1 \langle \text{grad} f(x+st), s\rangle dt && (38)\\
|f(x+s) - f(x) - \langle \text{grad} f(x), s\rangle| &= |\int_0^1 \langle \text{grad} f(x+st) - \text{grad} f(x), s\rangle dt| && (39)\\
&\leq \int_0^1 \|\text{grad} f(x+ts) - \text{grad} f(x)\|\|s\|dt && (40)\\
&\leq \|s\| \int_0^1 L\|ts\|dt && (41)\\
&= \|s\|^2 L[\frac{t^2}{2}]_0^1 && (42)\\
&= \|s\|^2 L[\frac{1}{2} - 0] && (43)\\
&= \frac{L}{2}\|s\|^2 && (44)\\
& && (45)
\end{aligned}
$$

Lipschitz continuity of the gradient is common to assume in Euclidean optimization problems, given the simple upper bound it obtains. When generalizing L-continuity to manifolds, its important to note that the gradient at two different points lie on different spaces. This is where parallel transport comes in, as it allows us to compare vectors on different tangent spaces. However, Assumption 3 allows us to bypass finding the parallel transport equation, making it easier to compute sufficient decrease.

Proposition 7. *Let f be a smooth function on a Riemannian manifold $\mathcal{M}$. For a retraction R, let $f \circ R$ satisfy Assumption 3 on $S \subseteq T\mathcal{M}$ with constant $L > 0$. If the pairs $\{(x_0, s_0), (x_1, s_1), (x_2, s_2), ...\}$ generated with step-sizes $\alpha_k \in [\alpha_{min}, \alpha_{max}] \subset (0, \frac{2}{L})$ all lie in S, then Algorithm 1 produces sufficient decrease. Specifically Assumption 2 holds with*

$$c = \min(\alpha_{min} - \frac{L}{2}\alpha_{min}^2, \alpha_{max} - \frac{L}{2}\alpha_{max}^2) > 0$$

In particular, for a constant $\alpha_k = \frac{1}{L}$, we have $c = \frac{1}{2L}$

Proof. By Assumption 3 in terms of the pullback, we can say for all $k \geq 0$

$$f(x_{k+1}) = f(R_{x_k}(s_k)) \leq f(x_k) + \langle \text{grad} f(x_k), s_k\rangle + \frac{L}{2}\|s_k\|^2$$

If we use $s_k = -\alpha_k \text{grad} f(x_k)$ from Algorithm 1, the equation becomes

$$
\begin{aligned}
f(x_{k+1}) &\leq f(x_k) + \langle \text{grad} f(x_k), s_k\rangle + \frac{L}{2}\|s_k\|^2\\
f(x_k) - f(x_{k-1}) &\leq \langle \text{grad} f(x_k), -\alpha_k \text{grad} f(x_k)\rangle + \frac{L}{2}\| - \alpha_k \text{grad} f(x_k)\|^2\\
&\leq -\alpha_k \|\text{grad} f(x_k)\|^2 + \alpha_k^2 \frac{L}{2}\|\text{grad} f(x_k)\|^2\\
&\leq (-\alpha_k + \alpha_k^2 \frac{L}{2})\|\text{grad} f(x_k)\|^2\\
&\geq (\alpha_k - \alpha_k^2 \frac{L}{2})\|\text{grad} f(x_k)\|^2
\end{aligned}
$$

The coefficient is quadratic in α_k, positive between roots at 0and$\frac{2}{L}$, By the assumption on α_k,

$$\alpha_k - \alpha_k^2 \frac{L}{2} \geq \min(\alpha_{min} - \frac{L}{2}\alpha_{min}^2, \alpha_{max} - \frac{L}{2}\alpha_{max}^2) > 0$$

□

2.4 Backtracking Method

An important result from the previous section comes from the earlier Proposition, where if we assume a constant step size $\alpha_k = 1/L$, we can show the pairs $\{(x_0, s_0), (x_1, s_1), ...\} \in S \subseteq T\mathcal{M}$, then the gradient approaches 0. However, it is very rare we know L, and if so, its possible that it could be very large due to a small domain in $f \circ R$. This leads to taking very small step-sizes, leading to longer convergence. To combat this, we can find optimal α_k for each iteration by looking at the local behavior at x_k.

Algorithm 2 Backtracking line-search method

Params: $\tau, r \in (0,1)$ (e.g., $\tau = \frac{1}{2},\ r = 10^{-4}$)
Input: $x \in \mathcal{M},\ \bar{\alpha} > 0$
Set $\alpha \leftarrow \bar{\alpha}$
while $f(x) - f(R_x(-\alpha \text{ grad } f(x_k))) < r\alpha \| \text{grad } f(x)\|^2$ **do**
  $\alpha \leftarrow \tau\alpha$
end while
Output: α

The backtracking line-search method works by setting an initial step size $\bar{\alpha}$, and iteratively reducing the step-size by a factor of $\tau \in (0,1)$ (often 0.8or0.5) until the below condition is satisfied.

$$f(x) - f(R_x(-\alpha \text{grad} f(x))) \geq r\alpha \|\text{grad} f(x)\|^2 \tag{46}$$

We can show that under Assumption 3, the backtracking line-search produces sufficient decrease (Assumption 2). The importance is that the value of L is not needed in order to guarantee convergence or run the algorithm. Relaxing our constraints allow us to apply RGD to many cases which we were not able to apply before.

Lemma 1. *Let f be a smooth function on $\mathcal{M}$. For a retraction R, a point $x \in \mathcal{M}$, and an initial step-size $\bar{\alpha} > 0$, let Assumption 3 hold for $f \circ R$ with constant L on $\{(x, -\alpha \text{grad} f(x) : \alpha \in [0, \bar{\alpha}]\}$. Then Algorithm 3 with parameters $\tau, r \in (0,1)$ outputs a step-size such that*

$$f(x) - f(R_x(\alpha \text{grad} f(x))) \geq r \min\left(\bar{\alpha}, \frac{2\tau(1-r)}{L}\right) \|\text{grad} f(x)\|^2 \tag{47}$$

after computing at most

$$\max\left(1, 1 + \log_{\tau^{-1}}\left(\frac{\bar{\alpha}L}{2(1-r)}\right)\right) \tag{48}$$

retractions and cost function evaluations (assuming $f(x)$ and $\text{grad} f(x)$ were already computed).

Proof. Assume $\text{grad} f(x) \neq 0$, as the converse trivially proves the claim. For all step-sizes α considered by Algorithm 2, Assumption 3's regularity condition tells us that

$$f(x) - f(R_x(-\alpha \text{grad} f(x))) \geq \alpha \|\text{grad} f(x)\|^2 - \frac{L}{2}\alpha^2 \|\text{grad} f(x)\|^2. \tag{49}$$

However, if Algorithm 2 does not return a certain α and the above conditions holds, then we can say:

$$\alpha \|\text{grad} f(x)\|^2 - \frac{L}{2}\alpha^2 \|\text{grad} f(x)\|^2 \leq f(x) - f(R_x(-\alpha \text{grad} f(x))) < r\alpha \|\text{grad} f(x)\|^2 \tag{50}$$

$$\alpha\|\text{grad}f(x)\|^2 - \frac{L}{2}\alpha^2\|\text{grad}f(x)\|^2 < r\alpha\|\text{grad}f(x)\|^2 \tag{51}$$

$$\alpha(1-r)\|\text{grad}f(x)\|^2 < \frac{L}{2}\alpha^2\|\text{grad}f(x)\|^2 \tag{52}$$

$$(1-r) < \frac{L}{2}\alpha \tag{53}$$

$$\alpha > \frac{2}{(1-r)L} \tag{54}$$

The inverse inequality gives us a lower bound for α before Algorithm 2 terminates and returns the optimal step-size. This can happen where $\bar{\alpha}$ may be less than the lower bound, or reducing α by a factor by τ drops it. Its important to note that this lower bound is not only point at which the algorithm can terminate. However, we conclude that the α returned by the algorithm satisfies:

$$\alpha \geq \min\left(\bar{\alpha}, \frac{2\tau(1-r)}{L}\right), \tag{55}$$

which proves the first condition. For the second condition, note that the returned α follows the form $\alpha = \bar{\alpha}\tau^{n-1}$ where n is the number of retractions and cost function evaluations computed by Algorithm 2. Then,

$$n = 1 + \log_\tau\left(\frac{\alpha}{\bar{\alpha}}\right) = 1 + \log_{\tau^{-1}}\left(\frac{\bar{\alpha}}{\alpha}\right) \leq 1 + \max\left(0, \log_{\tau^{-1}}\left(\frac{\bar{\alpha}L}{2(1-r)}\right)\right) \tag{56}$$

□

Commonly, we use the backtracking line-search algorithm in conjecture with the RGD algorithm. For this reason, we may want to dynamically pick our initial step-size $\bar{\alpha}$ as $\bar{\alpha}_k$ for some iteration k. As long as $\bar{\alpha}_k$ is bounded away from 0, then the convergence holds.

Corollary 1. *Let f be a smooth function satisfying Assumption 1 on $\mathcal{M}$. For a retraction R, let $f \circ R$ satisfy Assumption 3 on $S \subseteq T\mathcal{M}$ with constant L. Let $x_0, x_1, x_2, ...$ be the iterates generated by Algorithm 1 with Algorithm 2 using fixed parameters $\tau, r \in (0,1)$ and initial step-sizes $\bar{\alpha}_0, \bar{\alpha}_1, \bar{\alpha}_2, ...$ If for every k the set $\{(x_k, -\alpha\text{grad}f(x_k)) : \alpha \in [0, \bar{\alpha}_k]\}$ is in S and if $\liminf_{k\to\infty} \bar{\alpha}_k = 0$, then*

$$\lim_{k\to\infty} \|\text{grad}f(x_k)\| = 0 \tag{57}$$

Furthermore, $\forall\ K \geq 1,\ \exists k \in \{0, ..., K-1\}$ such that

$$\|\text{grad}f(x_k)\| \leq \sqrt{\frac{f(x_0) - f_{low}}{r\min(\bar{\alpha}_0, \bar{\alpha}_1, ..., \bar{\alpha}_{K-1}, \frac{2\tau(1-r)}{L}}}\frac{1}{\sqrt{K}}. \tag{58}$$

Proof. By the above lemma, the backtracking line-search algorithm guarantees sufficient decrease in the form of

$$f(x_k) - f(x_{k+1}) \geq c_k\|\text{grad}f(x_k)\|^2, \qquad c_k = r\min\left(\bar{\alpha}_k, \frac{2\tau(1-r)}{L}\right) \tag{59}$$

From a previous proposition 6, we can follow the same process to show

$$f(x_0) - f_{low} \geq \sum_{k=0}^{K-1} f(x_k) - f(x_{k+1}) \tag{60}$$

$$\geq \sum_{k=0}^{K-1} c_k\|\text{grad}f(x_k)\|^2 \tag{61}$$

$$\geq K \cdot \min_{k\in\{0,...,K-1\}} c_k \cdot \min_{k\in\{0,...,K-1\}} \|\text{grad}f(x_k)\|^2. \tag{62}$$

We can rearrange these terms to get us

$$\frac{f(x_0) - f_{low}}{Kr \min_{k\in\{0,...,K-1\}} c_k} \geq \|\text{grad} f(x_k)\|^2 \tag{63}$$

$$\|\text{grad} f(x_k)\| \leq \sqrt{\frac{f(x_0) - f_{low}}{Kr \min_{k\in\{0,...,K-1\}} c_k}} \tag{64}$$

$$= \sqrt{\frac{f(x_0) - f_{low}}{r \min\left(\bar{\alpha}_0, \bar{\alpha}_1, ..., \bar{\alpha}_k, \frac{2\tau(1-r)}{L}\right)}} \frac{1}{\sqrt{K}}, \tag{65}$$

proving our second condition. If we take $K \to \infty$, we get that $\lim_{k\to\infty} c_k \|\text{grad} f(x_k)\|^2 = 0$. Since $\liminf_{k\to\infty} c_k > 0$ (comes from our assumption), we can conclude that $\lim_{k\to\infty} \|\text{grad} f(x_k)\|^2 = 0$ □

2.5 Newton Method

In numerical analysis, there are a few ways of numerically evaluations roots of a function $f(x)$. A class of methods are called fixed point methods. These iterative methods create the sequence $\{x_k\}$ where $x_{k_1} = g(x_k)$. One of these fixed point methods is the Newton Method, with follows the iteration $x_{k+1} = g(x_k) = x_k - f(x_k)/f'(x_k)$ such that a root p holds $f(p) = 0$ and $f'(p) \neq 0$.

Theorem 1. *Let $f \in C^2[a, b]$ such that $p \in [a, b]$ and it is a root of $f(x)$, i.e. $f(p) = 0, f'(p) \neq 0$. Then $\exists\ \delta > 0$ such that the Newton Method generates a sequence $\{p_n\}_{n=0}^{\infty}$ that converges to $p \in [p - \delta, p + \delta]$.*

Proof. To prove the Newton Method generates a converging sequence, we need to show the 2 properties from the fixed point theorem hold. Those properties are that

1. $g(x) : [a, b] \to [a, b]$
2. $|g'(x)| \leq 1 \quad \forall x \in [a, b]$

Since $f \in C^2[a, b]$ and $f'(0) \neq 0$, there is a small neighborhood around p with radius δ such that $g(x)$ is continuously differentiable.

$$g'(x) = 1 - \frac{f'(x)^2 - f(x)f''(x)}{f'(x)^2} \tag{66}$$

$$= \frac{f'(x)^2 - f'(x)^2 - f(x)f''(x)}{f'(x)^2} \tag{67}$$

$$= \frac{f(x)f''(x)}{f'(x)^2} \tag{68}$$

$$= 0 \tag{69}$$

Because $g'(p) = 0$, that implies there is a small neighborhood with radius δ around p such that

$$|g'(x)| < 1 \qquad \forall x \in [p - \delta, p + \delta] \tag{70}$$

This proves the first condition of the fixed point iteration theorem. To show the second condition, let $x \in [p - \delta, p + \delta]$. Then since g is differentiable, we can use the Mean Value Theorem to show

$$|g(x) - g(p)| = |g'(\xi)||x - p| < |x - p| < \delta \tag{71}$$

This implies that $|g(x)| < \delta$ and that $g : [p - \delta, p + \delta] \to [p - \delta, p + \delta]$ □

In the context of optimization, gradient descent can also be interpreted as a fixed–point iteration applied to the equation $\nabla F(x) = 0$. Under this viewpoint, Newton's method corresponds to a *second–order gradient descent method*, since the update incorporates curvature information through the Hessian:

$$x_{k+1} = x_k - \left(\nabla^2 F(x_k)\right)^{-1} \nabla F(x_k). \tag{72}$$

We can extend this second–order gradient descent framework to the Riemannian setting as well. However, for brevity, we refer the reader to [15] or Chapter 5 of [2] for further details on second–order Riemannian gradient descent.

3 Constrained Convex Optimization Techniques

While the optimization algorithms we discussed are great and work extremely well in implementation. However there is one caveat with this procedure. For our algorithm, we have considered an unconstrained set, either it be $\mathbb{R}$ or $\mathcal{M}$. In many applications, we aim to find a minimum or maximum on a constrained set in some space, either $\mathbb{R}$ or $\mathcal{M}$. A typical problem of the sorts in $\mathbb{R}^d$ would look like

$$\min_{x \in S \subseteq \mathbb{R}^d} f(x), \tag{73}$$

where S is some convex feasible region capturing the constraints set on x and f is some objective function that typically satisfies smoothness and convexity. Regular gradient descent algorithms are not capable of handling these problems, as moving x_k may lead to a point that lies outside of the set. We will discuss various methods of how to go about this problem, and generalize the Euclidean setting to a Riemannian manifold setting.

3.1 Preliminary Methods

3.1.1 Projected Gradient Descent

There are some preliminary methods we will discuss to consider convex constraints. One of these methods being Projected Gradient Descent. The idea of this is to project our resultant value x_{k+1} from the gradient descent iteration. We can write this as two steps

$$x_{k+\frac{1}{2}} \to x_k - \alpha_k \text{grad} f(x_k) \tag{74}$$

and

$$x_{k+1} \to \Pi_S(x_{k+\frac{1}{2}}). \tag{75}$$

Here we let $\Pi_S(x)$ as the projection operator on x. We define this operator as $\Pi_S(x) \doteq \arg\min_{y \in S} \|x - y\|$. Note that if x is already in the convex set S, then the operator is just the identity mapping. One problem with this, is that computing the projection for each iteration is expensive, assuming the projection is known. In cases where S is a more simple convex set, PGD is an attractive solution. This is where an alternate solution comes in play.

3.1.2 Interior-Point Method

The Interior Point Method contrasts from the Projected Gradient Descent method because the iterates $\{x_0, x_1, x_2, ...\}$ strictly stay inside the convex set S. It does this by considering a barrier function which increases as $x \in S$ reaches the boundary of S. Instead of correcting for the constraint $x \in S$ explicitly, we define this barrier function as $B(x) : \text{int} S \to \mathbb{R}^d$ such that

1. $B(x)$ is smooth and strongly convex
2. $B(x) \to +\infty$ as $x \to \partial S$ where ∂S denotes the boundary of S

For a parameter $t > 0$, we can augment the original cost function to include this barrier function:

$$B_t(x) = tf(x) + B(x) \quad \text{with} \quad x^*(t) = \arg\min_{x \in \text{int}S} B_t(x) \tag{76}$$

Here, we can see that for each t, the minimizer x^* lies inside int$S$ because of the second condition of $B(x)$. As $t \to \infty$, the sequence $\{x_k\}$ trace a *central path* which converges to the solution of the original problem.

Both of these methods turn the constrained convex optimization problem into a unconstrained problem. Projected Gradient Descent does this by correcting updates that violate constraints and the Interior-Point Method penalizes nearing violations. While both these methods are well sufficient in smaller-scale convergence, there is another method that we will cover which will prove to be superior on larger-scale machine learning problems.

3.2 Frank-Wolfe Algorithm

The Frank-Wolfe algorithm does not turn the problem into an unconstrained problem, because it instead moves each iteration through convex combinations. Since we know our constrained set S is convex and compact, the iterates never leave S. This algorithm is considered a *projection-free* method.

Compared to the two previous methods, the FW algorithm requires far less computations compared to projections or Newton systems, making each iteration cheaper to compute. So when working with large-scale datasets, the FW method proves to be beneficial compared to the previous methods.

The Frank-Wolfe algorithm relies on the linear minimizing oracle (LMO), $y = \arg\min_{v \in S} \langle \text{grad} f(x), v \rangle$. This subroutine is iteratively solved, and combined with a convex combination, allows us to avoid projections. Compared to projection-based methods, which directly follow the negative gradient, the FW algorithm optimizes the negative gradient over our convex set. This vertex given from the LMO is then used to provide an alternate direction of descent. Noting that solving $y = \arg\min_{v \in S} \langle \text{grad} f(x), v \rangle$ is the same as $y = \arg\min_{v \in S} \langle \text{grad} f(x), v - x \rangle$, we can write the FW algorithm on Euclidean space as below.

Algorithm 3 Frank-Wolfe Algorithm on Euclidean Space

Input: Initial $x_0 \in S \subset \mathbb{R}^d$, f such that f is smooth and convex
for $k = 0, 1, ...$ **do**
  $v_k \leftarrow \arg\min_{v \in S} \left\langle \text{grad} f(x_k), v - x_k \right\rangle$
  $s_k \leftarrow \frac{2}{k+2}$
  $x_{k+1} \leftarrow (1 - s_k)x_k + s_k v_k$
end for

Its important to note that S must be convex and compact, in order to ensure $x_{k+1} \in S$ and

$$\arg\min_{z \in S} \left\langle \text{grad} f(x_k), v - x_k \right\rangle$$

has a solution. Also by setting our parameter $s_k = \frac{2}{2+k}$ takes away any sort of parameter tuning for our algorithm. This in turn also reduces computations necessary to find optimal step-sizes like line-search, however can still be implemented if needed.

To apply the Frank-Wolfe algorithm to the Riemannian manifold setting, we need to make a few modifications. From our proposition covering g-convex spaces, we need to replace the LMO with

$$v_k = \min_{v \in S} \left\langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(v) \right\rangle$$

and replace our linear convex combination with a geodesic, iterating with $x_{k+1} = \gamma(s_k)$ where γ is a geodesic with the initial conditions $\gamma(0) = x_k$ and $\gamma'(0) = v_k$.

For our convergence and complexity analysis, we will stick to the Riemannian manifold environment, as the Euclidean space is a trivial case.

Algorithm 4 Frank-Wolfe Algorithm on Riemannian Manifolds

Input: Initial $x_0 \in S \subset \mathcal{M}, f$ such that f is smooth and convex
for $k = 0, 1, ...$ **do**
  $v_k \leftarrow \arg\min_{v \in S} \langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(v) \rangle$
  $s_k \leftarrow \frac{2}{k+2}$
  $x_{k+1} \leftarrow \gamma(s_k)$
end for

3.2.1 Convergence and Complexity

Before we prove convergence, we need to introduce the *curvature constant*, which plays an important role in the convergence rate. We define the curvature constant as

$$M_f := \sup_{x,y,z \in S} \frac{2}{\eta^2} \Big[f(y) - f(x) - \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \Big\rangle_x \Big]. \tag{77}$$

For notation, let $y = c(\eta)$ for some $\eta \in [0, 1]$ and the geodesic $c : [0, 1] \to \mathcal{M}$ where $c(0) = x$ and $c(1) = z$. We will refer to this specific geodesic as $c_{x,z}$ in future proofs. In the Euclidean setting, a comparable constant is used, in which the traditional gradient and Euclidean log map are used. Another note is that many of the proofs are adapted from the Euclidean FW algorithm.

Lemma 2. *Let $f : \mathcal{M} \to \mathbb{R}$ be L-smooth on S and let $\text{diam}(S) := \sup_{x,y \in S} d(x, y)$. Then, the curvature constant M_f satisfies the bound $M_f \leq L \ \text{diam}(S)^2$.*

Proof. Let $x, z \in S$ and $\eta \in (0, 1)$. Also let $y = c_{x,z}(\eta)$ be a point on the geodesic joining x with z. Because the total length of a geodesic is $d(x, z)$, we can say that $d(x, c_{x,z}(t)) = |0 - t| d(x, z)$. Now if we let $t = \eta$, then we get $d(x, y) = \eta d(x, z)$. Squaring both sides and isolating $d(x, z)$ gives us $\frac{1}{\eta^2} d(x, y)^2 = d(x, z)^2$. Using the definition of *L-smooth* functions, we can say that

$$\Big| f(y) - f(x) - \big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \big\rangle_x \Big|^2 \leq \frac{L}{2} d(x, y)^2 \tag{78}$$

and now using the definition of the curvature constant, we can say

$$M_f \leq \sup_{x,y,z \in S} \frac{2}{\eta^2} \frac{L}{2} d(x, y)^2 = \sup_{x,y,z \in S} L \ d(x, z)^2 \leq L \ \text{diam}(S)^2. \tag{79}$$

□

Lemma 3. *(Lipschitz) Let $x, y, z \in S$ and $\eta \in [0, 1]$ with $y = c_{x,z}(\eta)$. Then,*

$$f(y) \leq f(x) + \eta \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(z) \Big\rangle_x + \frac{1}{\eta^2} M_f. \tag{80}$$

Proof. Because M_f is the supremum, we can say that

$$M_f \geq \frac{2}{\eta^2} \Big[f(y) - f(x) - \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \Big\rangle_x \Big] \tag{81}$$

$$\frac{2}{\eta^2} f(y) \leq M_f - \frac{2}{\eta^2} \Big(f(x) - \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \Big\rangle_x \Big) \tag{82}$$

$$f(y) \leq \frac{\eta^2}{2} + f(x) + \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \Big\rangle_x. \tag{83}$$

Since $y = c_{x,z}(\eta)$, we can say that $\text{Exp}_x^{-1}(y) = \eta \text{Exp}_x^{-1}(z)$. Therefore,

$$\Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(y) \Big\rangle_x = \Big\langle \text{grad} f(x), \eta \text{Exp}_x^{-1}(z) \Big\rangle_x = \eta \Big\langle \text{grad} f(x), \text{Exp}_x^{-1}(z) \Big\rangle_x \tag{84}$$

Plugging this into our rewritten equation concludes the proof. □

Lemma 4. *Let $(a_k)_{k\in I}$ be a nonnegative sequence that fulfills*

$$a_{k+1} \leq (1 - s_k)a_k + \frac{1}{2}s_k^2 M_f. \tag{85}$$

If $s_k = \frac{2}{k+2}$, then $a_k \leq \frac{2M_f}{k+2}$.

Proof. We can prove this lemma through induction.

Base Case: $k = 0$

$$a_1 \leq (1 - \frac{2}{0+2})a_0 + \frac{1}{2}\Big(\frac{2}{0+2}\Big)^2 M_f = (0)a_0 + M_f = M_f \tag{86}$$

Induction Hypothesis: Assume for some fixed k, if $s_k = \frac{2}{k+2}$, then $a_k \leq \frac{2M_f}{k+2}$.

Induction Step: For $k + 1$,

$$\begin{aligned}
a_{k+1} &\leq \Big(1 - \frac{2}{k+2}\Big)a_k + \frac{1}{2}\Big(\frac{2}{k+2}\Big)^2 M_f \\
&\leq \Big(1 - \frac{2}{k+2}\Big)\frac{2M_f}{k+2} + \frac{1}{2}\Big(\frac{2}{k+2}\Big)^2 M_f \\
&= \frac{2M_f}{k+2} - \frac{4M_f}{(k+2)^2} + \frac{2M_f}{(k+2)^2} \\
&= \frac{2M_f}{k+2} - \frac{2M_f}{(k+2)^2} \\
&= \frac{2(k+2)M_f - 2M_f}{(k+2)^2} \\
&= \frac{2(k+1)M_f}{(k+2)^2} \\
&= \frac{2(k+1)M_f}{k^2+4k+4} \\
&\leq \frac{2(k+1)M_f}{k^2+4k+3} \\
&= \frac{2(k+1)M_f}{(k+3)(k+1)} \\
&= \frac{2M_f}{k+3}
\end{aligned}$$

□

With these three lemmas, we have the mechanics to now prove sub-linear convergence $O(\frac{1}{k})$ of our RFW algorithm.

Theorem 2. *Let $s_k = \frac{2}{2+k}$ and let x^* be a minimum of f. Then, the sequence of iterates $\{x_k\}$ generated by Algo 4 satisfies $f(x_k) - f(x^*) = O(\frac{1}{k})$*

Proof. The proof comes from Lemmas 2, 3, 4;

$$\begin{aligned}
f(x_{k+1}) - f(x^*) &\leq f(x_k) - f(x^*) + s_k\Big\langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(v_k)\Big\rangle + \frac{1}{2}s_k^2 M_f \\
&\leq f(x_k) - f(x^*) + s_k\Big\langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(x^*)\Big\rangle + \frac{1}{2}s_k^2 M_f \\
&\leq f(x_k) - f(x^*) + s_k\Big(f(x_k) - f(x^*)\Big) + \frac{1}{2}s_k^2 M_f \\
&\leq (1 - s_k)\Big(f(x_k) - f(x^*)\Big) + \frac{1}{2}s_k M_f \\
&\leq (1 - \frac{2}{k+2})\Big(f(x_k) - f(x^*)\Big) + \frac{1}{2}\frac{2}{k+2}M_f \\
f(x_k) - f(x^*) &\leq \frac{2M_f}{k+2}
\end{aligned}$$

where the first inequality comes from Lemma 3, and the second one comes from the fact that the algorithm find the z_k argmin. The third inequality comes from the definition of a geodesically convex function. By setting $a_k = f(x_k) - f(x^*)$, we can use Lemma 4 to come to our final inequality, providing us with our desired $O(\frac{1}{k})$. □

This sublinear convergence rate is due to the nature of the Frank-Wolfe method, which is discussed in [4], which mentions the "zig-zag" nature of the convergence, which occurs when the minimum lies on the boundary of S. However, many papers also highlight the possibility of a linear convergence given the optimal solution is an interior point. This assumption will not guarantee a global minimum that we get with the sub-linear convergence. The popular Polyak-Łojasiewicz (PL) inequality has been used many times to show linear convergence with gradient methods. Consider the unconstrained manifold optimization problem

$$\min_{x \in \mathcal{M}} f(x),$$

with f^* as the optimal value. Then the PL inequality states that

$$\frac{1}{2}\|\text{grad} f(x)\|^2 \geq \mu(f(x) - f^*) \qquad \mu > 0, \forall\ x, y \in \mathcal{M} \tag{87}$$

Theorem 3. *Suppose that f is strongly g-convex with constant μ and that its minimum lies in a ball of radius that is strictly inside the constraint set S. Define $\Delta_k := f(x_k) - f(x^*)$ and let the step-size $s_k = \frac{r\sqrt{\mu\Delta_k}}{\sqrt{2M_f}}$. Then, RFW converges linearly and satisfies*

$$\Delta_{k+1} \leq \Big(1 - \frac{3r^2\mu}{4M_f}\Big)\Delta_k \tag{88}$$

Proof. Let $\mathcal{B}_r(x^*) \subset S$ be a ball of radius r containing the optimum. Define

$$W_k := \text{argmax}_{\substack{\xi \in T_{x_k}\mathcal{M} \\ \|\xi\| \leq 1}} \langle \xi, \text{grad} f(x_k)\rangle$$

be the direction of steepest descent in the tangent space $T_{x_k}\mathcal{M}$. The point $P_k = \text{Exp}_{x_k}(rW_k)$ lies in S. We can rewrite the following equality

$$\Big\langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(P_k)\Big\rangle = -\langle \text{grad} f(x_k), rW_k\rangle = -r\|\text{grad} f(x_k)\| \tag{89}$$

given the way we define W_k. Note that the first inner product comes from Step 3 of the Algo 4. Thus using the Lipschitz bound assumption of RFW, we can show

$$\begin{aligned}
\Delta_{k+1} &\leq \Delta_k + s_k\Big\langle \text{grad} f(x_k), \text{Exp}_{x_k}^{-1}(x_{k+1})\Big\rangle + \frac{1}{2}s_k^2 M_f, \\
&\leq \Delta_k - s_k r\|\text{grad} f(x_k)\| + \frac{1}{2}s_k^2 M_f, \qquad \text{by(89)} \\
&\leq \Delta_k - \sqrt{2\mu}\sqrt{f(x_k) - f(x^*)} + \frac{1}{2}s_k^2 M_f, \qquad \text{by(87)} \\
&\leq \Delta_k - \sqrt{2\mu}\sqrt{\Delta_k} + \frac{1}{2}s_k^2 M_f.
\end{aligned}$$

Now setting the step-size $s_k = \frac{r\sqrt{\mu\Delta_k}}{\sqrt{2}M_f}$, we get

$$\begin{aligned}
\Delta_{k+1} &\leq \Delta_k - \frac{r\sqrt{\mu\Delta_k}}{\sqrt{2}M_f} r\sqrt{2\mu}\sqrt{\Delta_k} + \frac{1}{2}\left(\frac{r\sqrt{\mu\Delta_k}}{\sqrt{2}M_f}\right)^2 M_f, \\
&\leq \Delta_k - \frac{r^2\mu\Delta_k}{M_f} + \frac{1}{2}\frac{r^2\mu\Delta_k}{2M_f^2} M_f \\
&\leq \Delta_k - \frac{r^2\mu\Delta_k}{M_f} + \frac{r^2\mu\Delta_k}{4M_f} \\
&\leq \Delta_k - \frac{3r^2\mu\Delta_k}{4M_f} \\
&\leq \Delta_k\Big(1 - \frac{3r^2\mu}{4M_f}\Big)
\end{aligned}$$

Which concludes the proof □

4 Application

With the methods we have discussed, we will now discuss the application where Riemannian optimization methods can be applied. For brevity, two applications will be discussed, however refer to [1] and [3] for more applications. Its important to note, per example, there must be a well-defined exponential map, gradient, and logarithmic map.

4.1 Principal Component Analysis (PCA)

Commonly, when dealing with high dimensional data, the variance of the data is approximately concentrated along a lower-dimensional subspace. A frequent question in data analysis is to understand the main directions of variance in data. This is the basis of the PCA idea.

Let $x_1, ..., x_n \in \mathbb{R}^d$ represent a large collection of centered data points with d features. We can define a matrix $X \in \mathbb{R}^{d\times n}$ holding these data points. The goal of our problem is to find a set of k principal components, or orthonormal vectors such that we can maximize the following:

$$f(U) = \langle XX^\top U, U\rangle_F = \text{tr}(U^\top XX^\top U) \qquad U \in \mathbb{R}^{d\times k}, \tag{90}$$

As we can see, the matrix $S = XX^\top \in \mathbb{R}^{d\times d}$ is proportional to the sample covariance matrix, where $S_{i,j}$ measures how feature $i$and$j$ co-vary with our data X. The function $f(U)$ tells us the total variance projected onto the subspace spanned by$U$. The set of such matrices U is called the *Stiefel manifold*,

$$\text{St}(d,k) = \{U \in \mathbb{R}^{d\times k} : U^\top U = I_k\},$$

which is a smooth Riemannian manifold embedded in $\mathbb{R}^{d\times k}$ where the "orthonormaility" constraint $U^\top U = I_k$ makes this a constrained Riemannian optimization problem. Here, the matrix $U* = \arg\max_{U\in\text{St}(d,k)} f(U)$

represents the top k orthonormal eigenvectors that span a subspace to maximize the variance that is preserved after projecting. Before we can run algorithm 1, we need to compute the gradient and retraction equation.

4.1.1 PCA Computation

Gradient: The first step to computing the gradient on $\mathrm{St}(d,k)$ is to calculate the smooth extension through equation (1).

$$\begin{aligned}
Df(U)[V] &= \lim_{t\to 0} \frac{f(U+Vt)-f(U)}{t} \\
&= \lim_{t\to 0} \frac{\langle (U+Vt)^\top XX^\top, (U+Vt)\rangle_F - \langle X^\top XU, U\rangle_F}{t} \\
&= \lim_{t\to 0} \frac{\langle U^\top XX^\top, U\rangle_F + \langle tV^\top XX^\top, tV\rangle_F + \langle U^\top XX^\top, tV\rangle_F + \langle tV^\top XX^\top, U\rangle_F - \langle X^\top XU, U\rangle_F}{t} \\
&= \lim_{t\to 0} \frac{\langle U^\top XX^\top, U\rangle_F + t^2\langle V^\top XX^\top, V\rangle_F + t\langle U^\top XX^\top, V\rangle_F + t\langle V^\top XX^\top, U\rangle_F - \langle X^\top XU, U\rangle_F}{t} \\
&= \lim_{t\to 0} \frac{t^2\langle V^\top XX^\top, V\rangle_F + t\langle U^\top XX^\top, V\rangle_F + t\langle V^\top XX^\top, U\rangle_F}{t} \\
&= \lim_{t\to 0} t\langle V^\top XX^\top, V\rangle_F + \langle U^\top XX^\top, V\rangle_F + \langle V^\top XX^\top, U\rangle_F \\
&= \langle U^\top XX^\top, V\rangle_F + \langle V^\top XX^\top, U\rangle_F \\
&= \langle U^\top XX^\top, V\rangle_F + \langle U^\top X^\top X, V\rangle_F \\
&= 2\langle U^\top XX^\top, V\rangle_F
\end{aligned}$$

giving us $\mathrm{grad}\bar{f}(U) = 2XX^\top U$. Now we need to project it to the manifold with the notion of the tangent space $T_U\mathrm{St}(d,k) = \{Z : U^\top Z + Z^\top U = 0\}$. We can define the projection as a minimization problem:

$$\mathrm{Proj}_U(G) = \arg\min_Z \|G-Z\|_F^2 \qquad G = \mathrm{grad}f(U) \tag{91}$$

which can be written as a Lagrange multiplier problem $\mathcal{L}(Z,A) = \|G-Z\|_F^2 + \langle A, U^\top Z + Z^\top U\rangle_F$ where $A \in \mathbb{R}^{k\times k}$ is symmetric. Solving for a critical point guarantees us a minimum because $\mathcal{L}$ is convex since the determinant of the Hessian is greater than 0 (in particular, $\mathrm{Hess}\mathcal{L} = 2I_k$).

$$\frac{\partial \mathcal{L}}{\partial Z} = -2(G-Z) + 2UA = 0 \tag{92}$$

$$-2G + 2Z + 2UA = 0 \tag{93}$$

$$2Z = 2G - 2UA \tag{94}$$

$$Z^* = G - UA. \tag{95}$$

Plugging this into our condition of the tangent space, we get:

$$\begin{aligned}
0 &= U^\top(G-UA) + (G-UA)^\top U \\
&= U^\top G - U^\top UA + G^\top U - A^\top U^\top U \\
&= U^\top G - A + G^\top U - A^\top \\
&= U^\top G + G^\top U - 2A \\
2A &= U^\top G + G^\top U \\
A &= \frac{U^\top G + G^\top U}{2}
\end{aligned}$$

The final step to finding the projected gradient on the Stiefel manifold is plugging A into the critical point (95).

$$\begin{aligned}
\text{Proj}_U(G) &= Z && (96)\\
&= G - U\frac{U^\top G + G^\top U}{2} && (97)\\
&= 2XX^\top U - U\frac{U^\top(2XX^\top U) + (2XX^\top U)^\top U}{2} && (98)\\
&= 2XX^\top U - U\frac{2U^\top XX^\top U + 2U^\top XX^\top U}{2} && (99)\\
&= 2XX^\top U - 2UU^\top XX^\top U && (100)\\
&= 2(I_d - UU^\top)XX^\top U && (101)
\end{aligned}$$

Retraction: Consider a point $U \in \text{St}(d,k)$ with orthonormal columns and a tangent vector $Z \in T_U\text{St}(d,k)$. For small t, the matrix $U+tZ \in \mathbb{R}^{d\times k}$ is generally *not* on the Stiefel manifold since $(U+tZ)^\top(U+tZ) \neq I_k$. To combat this, we can define a retraction via the QR decomposition:

$$U + tZ = Q(t)R(t), \tag{102}$$

where $Q(t)$ has orthonormal columns and $R(t)$ is upper triangular. The retraction is then

$$R_U(tZ) = Q(t).$$

A valid retraction must satisfy:

$$R_U(0) = U, \qquad \frac{d}{dt}R_U(tZ)\Big|_{t=0} = Z.$$

Initial Condition 1:
At $t = 0$:

$$U + 0 = U \cdot I_k = Q(0)R(0),$$

where we can say $Q(0) = U \text{and} R(0) = I_k$. This gives us $R_U(0) = Q(0) = 0$, satisfying initial condition 1. For the second initial condition, we need to differentiate (102) with respect to t, giving us

$$\begin{aligned}
\frac{d}{dt}\Big(U + tZ\Big)\Big|_{t=0} &= \frac{d}{dt}\Big(Q(t)R(t)\Big)\Big|_{t=0}\\
Z &= \dot{Q}(0)R(0) + Q(0)\dot{R}(0)\\
&= \dot{Q}(0) + U\dot{R}(0)\\
U^\top Z &= U^\top\dot{Q}(0) + U^\top U\dot{R}(0)\\
&= U^\top\dot{Q}(0) + \dot{R}(0)\\
\dot{R}(0) &= U^\top Z - U^\top\dot{Q}(0)
\end{aligned}$$

To understand what $U^\top\dot{Q}(0)$ is, we can differentiate $Q(t)^\top Q(t) = I_k$ with respect to t, which will show that $U^\top\dot{Q}(0)$ is actually skew-symmetric, like $U^\top Z$.

$$\begin{aligned}
\frac{d}{dt}\Big(Q(t)^\top Q(t)\Big)\Big|_{t=0} &= \frac{d}{dt} I_k \Big|_{t=0} \\
\dot{Q}(0)^\top Q(0) + Q(t)^\top \dot{Q}(0) &= 0 \\
\dot{Q}(0)^\top U + U^\top \dot{Q}(0) &= 0 \\
U^\top \dot{Q}(0) &= -\dot{Q}(0)^\top U \\
&= -(U^\top \dot{Q}(0))^\top
\end{aligned}$$

Since $R(t)$ is upper triangular for all t, its derivative $R'(0)$ must also be upper triangular. But from the previous computation, $R'(0)$ must also be skew-symmetric:

$$R'(0) + R'(0)^\top = 0.$$

The only matrix that is both upper triangular and skew-symmetric is the zero matrix, meaning $\dot{R}(0) = 0 = U^\top Z - U^\top \dot{Q}(0)$ and $\dot{Q}(0) = Z$. Plugging this back into our condition gives

$$\frac{d}{dt} R_U(tZ)\Big|_{t=0} = \dot{Q}(0) = Z \tag{103}$$

proving the second condition holds. We can now say the retraction (102) is valid on the Stiefel manifold.

4.1.2 Simulation

We now have the mechanics to apply the gradient equation (101) and retraction equation (102) to the RGD algorithm. To demonstrate the accuracy of the algorithm, we will compare it to `sklearn.decomposition.PCA`, providing us a benchmark. As for the data, we can use the readily available `iris` dataset on `sklearn`, which contains 4 features:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

For the purpose of this simulation, we will assume our manifold is $\mathrm{St}(d = 4, k = 2)$, following both the RGD and Backtracking line search algorithms discussed. To define an error between our PCA code and the benchmark, we can refer to the subspace error norm. For a subspace U and the true subspace U^* (from `sklearn.decomposition.PCA`) , we can define the error as

$$E(U) = \|UU^\top - U^*U^{*\top}\|_F, \tag{104}$$

where $E(U) \to 0$ as the projections $UU^\top$ and $U^*U^{*\top}$ produce the same subspace. Essentially, this means that U and U^*, regardless of their columns, span the same subspace. After running the code found in Appendix 5.1, the subspace error $E(U)$ was $2.98 \cdot 10^{-9}$. This shows our PCA code produces the same result as the `sklearn.decomposition.PCA`.

4.2 Geometric Matrix Mean

In many applications of statistical inference, computing averages of input quantities is essential. When data lies in Euclidean space, this is straightforward. However, data in many problems—including portfolio selection, computer vision, and radar sensing—lies on non-Euclidean spaces. Embedding such data into Euclidean space risks destroying the underlying geometric structure, including convexity properties intrinsic to the manifold. Instead, we maintain the data in its natural space and apply Riemannian optimization tools

directly. The specific problem we consider is computing the *geometric matrix mean* over a set of Hermitian positive definite (HPD) matrices.

The space of $n \times n$ Hermitian matrices is defined as

$$\mathbb{H}_n = \{A \in \mathbb{C}^{n\times n} : A = A^H\}, \tag{105}$$

where $A^H = \bar{A}^\top$ denotes the conjugate transpose. A matrix $A \in \mathbb{H}_n$ is Hermitian positive definite (HPD) if $x^H Ax > 0$ for all nonzero $x \in \mathbb{C}^n$, written $A \succ 0$. The space of HPD matrices is denoted $\mathbb{P}_n$. Although $\mathbb{P}_n$ is an open subset of $\mathbb{H}_n$, it is not a subspace, as it forms an open convex cone and carries a natural Riemannian manifold structure with the affine-invariant metric. For constrained optimization we consider the set

$$S = \{A \in \mathbb{P}_n : L \preceq A \preceq U\}, \quad L,\, U \in \mathbb{P}_n, \tag{106}$$

for the remainder of this application.

Given matrices $A_1, \ldots, A_m \in \mathbb{P}_n$ with weights $w_i > 0$ satisfying $\sum_{i=1}^m w_i = 1$, the geometric matrix mean is defined as the Fréchet mean with respect to the geodesic distance on $\mathbb{P}_n$:

$$G := \arg\min_{X\in S} \left[f(X) = \sum_{i=1}^m w_i\, \delta_R^2(X, A_i) \right], \tag{107}$$

where

$$\delta_R(X, Y) = \left\| \log\Big(X^{-1/2} Y X^{-1/2} \Big) \right\|_F \tag{108}$$

is the Riemannian metric on $\mathbb{P}_n$, and $\log(\cdot)$ denotes the matrix logarithm.

4.2.1 Computations on HPD matrices

Gradient: Using the symmetry $\delta_R^2(X, A_i) = \| \log(A_i^{-1/2} X A_i^{-1/2})\|_F^2$, write f as

$$f(X) = \sum_{i=1}^n w_i \| \log(A_i^{-1/2} X A_i^{-1/2})\|_F^2. \tag{109}$$

For each i, let $h_i(X) = \| \log(A_i^{-1/2} X A_i^{-1/2})\|_F^2$. Setting $\varphi_1(X) = \varphi_2(X) = \log(A_i^{-1/2} X A_i^{-1/2})$, we have $h_i(X) = \langle \varphi_1(X), \varphi_2(X)\rangle_F$, so by the product rule:

$$Dh_i(X)[Y] = \langle D\varphi_1(X)[Y],\, \varphi_2(X)\rangle_F + \langle \varphi_1(X),\, D\varphi_2(X)[Y]\rangle_F. \tag{110}$$

From Chapter 2 of *Positive Definite Matrices* [9], We choose an orthonormal basis where X = diag$(\lambda_1, ..., \lambda_n)$

$$D\varphi_1(X)[Y] = U \left[\left[\frac{\log(\lambda_k) - \log(\lambda_j)}{\lambda_k - \lambda_j} \right] \right] \circ Y, \tag{111}$$

where $\circ$ denotes the Schur product. Using this,

$$\langle D\varphi_1(X)[Y],\, \varphi_2(X)\rangle_F = \langle \varphi_1'(X)\, \varphi_2(X),\, Y\rangle_F,$$

and similarly $\langle \varphi_1(X), D\varphi_2(X)[Y]\rangle_F = \langle \varphi_1(X)\, \varphi_2'(X), Y\rangle_F$, so that

$$Dh_i(X)[Y] = 2 \left\langle (A_i^{-1/2} X A_i^{-1/2})^{-1} \log(A_i^{-1/2} X A_i^{-1/2}),\; A_i^{-1/2} Y A_i^{-1/2} \right\rangle_F.$$

Expanding via the trace and using the two facts:

$$\mathrm{tr}(ABC) = \mathrm{tr}(BCA) \quad \text{and} \quad S \log(T) S^{-1} = \log(STS^{-1}) \tag{112}$$

$$
\begin{aligned}
Df(X)[Y] &= \sum_i w_i \cdot 2\,\mathrm{tr}\Big(A_i^{-1/2} X^{-1} A_i^{1/2} \log(A_i^{-1/2} X A_i^{-1/2})\, A_i^{-1/2} Y A_i^{-1/2}\Big) \\
&= \sum_i w_i \cdot 2\,\mathrm{tr}\Big(\log(A_i^{-1/2} X A_i^{-1/2})\, A_i^{-1/2} Y A_i^{-1/2} A_i^{-1/2} X^{-1} A_i^{1/2}\Big) \\
&= \sum_i w_i \cdot 2\,\mathrm{tr}\Big(\log(A_i^{-1/2} X A_i^{-1/2})\, A_i^{-1} Y X^{-1}\Big) \\
&= \sum_i w_i \cdot 2\,\mathrm{tr}\Big(X^{-1} A_i^{1/2} \log(A_i^{-1/2} X A_i^{-1/2})\, A_i^{-1} Y\Big) \\
&= \sum_i w_i \cdot 2\,\mathrm{tr}\big(X^{-1} \log(X A_i^{-1})\, Y\big) \\
&= \left\langle 2 \sum_i w_i X^{-1} \log(X A_i^{-1}),\ Y \right\rangle_F ,
\end{aligned}
$$

which identifies the Euclidean gradient as

$$
\nabla_H f(X) = 2 \sum_{i=1}^{n} w_i\, X^{-1} \log(X A_i^{-1}). \tag{113}
$$

Since $\mathcal{P}_n$ is an open subset of $\mathbb{H}_n$, we have $T_X \mathcal{P}_n \cong \mathbb{H}_n$. The Riemannian gradient $\mathrm{grad}\, f(X) \in T_X \mathcal{P}_n$ is defined by

$$
g_X(\mathrm{grad}\, f(X),\, V) = Df(X)[V] \qquad \forall\, V \in \mathbb{H}_n, \tag{114}
$$

where $g_X(U, V) = \mathrm{tr}(X^{-1} U X^{-1} V)$ is the affine-invariant metric. Setting $G = \mathrm{grad}\, f(X)$, this requires

$$
\mathrm{tr}(X^{-1} G\, X^{-1} V) = \mathrm{tr}(\nabla_H f(X) \cdot V) \qquad \forall\, V \in \mathbb{H}_n. \tag{115}
$$

By non-degeneracy of the trace inner product, $X^{-1} G X^{-1} = \nabla_H f(X)$, and multiplying on the left and right by X gives

$$
\mathrm{grad}\, f(X) = X\, \nabla_H f(X)\, X = 2 \sum_{i=1}^{n} w_i \log(X A_i^{-1})\, X. \tag{116}
$$

Inverse Exponential Map: To find the inverse of the exponential map, we will first need to calculate the geodesic equation. Doing this requires far more mechanics than what this paper can inform the reader, so for any further help, refer to [9]. However, we will still cover a sketch of the derivation of the parameterized geodesic map.

Theorem 4. *Let X and Y be any two elements on $\mathbb{P}_n$. Then there exists a unique geodesic $[X, Y]$ joining X and Y. This geodesic has a parameterization*

$$
\gamma(t) = X^{\frac{1}{2}} \Big(X^{-\frac{1}{2}} Y X^{-\frac{1}{2}}\Big)^t X^{\frac{1}{2}}, \qquad 0 \le t \le 1. \tag{117}
$$

Proof. Note that the matrices I and $X^{-\frac{1}{2}} Y X^{-\frac{1}{2}}$ commute, as in $X^{-\frac{1}{2}} Y X^{-\frac{1}{2}} I = I X^{-\frac{1}{2}} Y X^{-\frac{1}{2}}$. So the geodesic $[I, X^{-\frac{1}{2}} Y X^{-\frac{1}{2}}]$ is naturally parameterized as

$$
\gamma_0(t) = \Big(X^{-\frac{1}{2}} Y X^{-\frac{1}{2}}\Big)^t.
$$

This comes from the fact that if we have a parameterization $\bar{\gamma}$ that joins X and Y and has the same length as γ, we can denote a function $X(t) = \log \bar{\gamma}(t)$ which joins $\log X$ and $\log Y$. But since this lies on Euclidean space, the geodesic is a line segment connecting the two. Then we rewrite the parameterization as $X(t) = (1-t)\log(X) + t \log(B)$, giving us $\gamma(t) = X^{1-t} Y^t$. We can now apply a reparameterization using the isometry $\Gamma_X(A) = XAX$ and obtain

$$\gamma(t) = \Gamma_{X^{\frac{1}{2}}}\left(\gamma_0(t)\right) = X^{\frac{1}{2}}\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right)^t X^{\frac{1}{2}}.$$

This geodesic joins the points

$$\Gamma_{X^{\frac{1}{2}}}(I) = X \qquad \text{and} \qquad \Gamma_{X^{\frac{1}{2}}}\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right) = Y,$$

meaning that since $\Gamma_{X^{\frac{1}{2}}}$ is an isometry, this path is actually the geodesic connecting X and Y, regardless if X and Y commute. □

Now we can find the inverse exponential map by differentiating the geodesic equation we have sketched out in the above theorem. However, to differentiate the geodesic, we will need a lemma.

Lemma 5. *Define $M := X^{-\frac{1}{2}}YX^{-\frac{1}{2}}$, then because M is a semi-positive definite (SPD) matrix, we can say*

$$M^t = \exp(t\log(M)) \tag{118}$$

The proof follows from that we can diagonalize any SPD, and then we take the matrix logarithm of M. Lastly, we would define a matrix $S := \log(M)$ and take the matrix exponential parameterized by t. Now we can differentiate the geodesic to get the inverse exponential map.

From Lemma 5,

$$\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right)^t = \exp\left(t\log\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right)\right). \tag{119}$$

So

$$\gamma(t) = X^{\frac{1}{2}}\exp(tS)X^{\frac{1}{2}}, \qquad \text{where } S := \log\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right)$$

Differentiating gives us

$$\gamma'(t) = X^{\frac{1}{2}}\exp(tS)SX^{\frac{1}{2}},$$

and evaluating at $t = 0$ and defining it as our velocity matrix Z gives us the inverse exponential map.

$$\gamma'(0) = Z = X^{\frac{1}{2}}\log\left(X^{-\frac{1}{2}}YX^{-\frac{1}{2}}\right)X^{\frac{1}{2}} \tag{120}$$

This comes from the fact that we start with the geodesic joining X and Y and we found the velocity of the geodesic at $t = 0$, which is the definition of the inverse exponential map.

4.2.2 Simulation

To illustrate the convergence behavior of the Riemannian Frank–Wolfe algorithm on the manifold of Hermitian positive definite matrices, we conduct a numerical experiment with $N = 100$ matrices of size $n = 5$. We generate HPD matrices by sampling $A_i \in \mathbb{R}^{n\times n}$ with independent uniform entries and forming

$$A_i \leftarrow A_i A_i^\top + 10^{-3}I_n, \tag{121}$$

ensuring strict positive definiteness. Each matrix is assigned a random weight $w_i \geq 0$, and the weights are normalized so that $\sum_{i=1}^N w_i = 1$.To obtain a high-accuracy approximation of the weighted Riemannian geometric mean, we run a Riemannian gradient descent procedure for 500 iterations:

$$X_{k+1} = X_k \#_{1.0}\left(X_k - \eta\, X_k^{1/2}\nabla f(X_k)X_k^{1/2}\right), \tag{122}$$

with a small step-size $\eta = 0.01$. This produces a reference point X_{true} that is used to evaluate the convergence error of RFW. We initialize RFW at $X_0 = A_1$ and run Algorithm 4 while tracking the Riemannian distance $\delta_R(X_k, X_{\text{true}})$ at each iteration. The resulting error curve is shown in Figure 1. The distance decreases monotonically, demonstrating the expected sublinear convergence behavior of Frank–Wolfe on the HPD manifold.

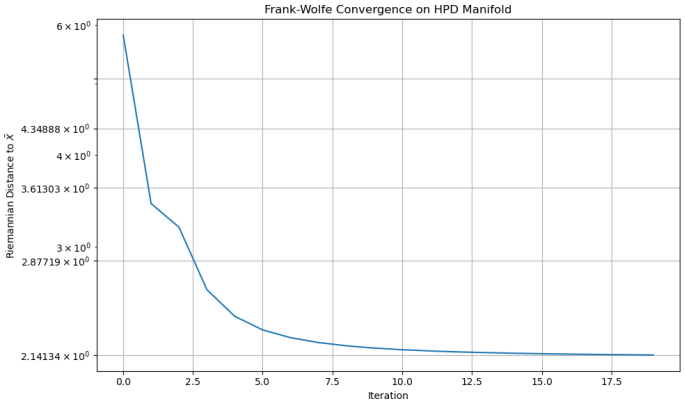


Figure 1: Convergence of the Riemannian Frank–Wolfe algorithm for computing the weighted geometric mean of $N = 100$ HPD matrices of size $n = 5$.

References

[1] M. Weber and S. Sra, "Riemannian Optimization via Frank-Wolfe Methods," *Mathematical Programming*, vol. 199, pp. 395–427, 2022. DOI: 10.1007/s10107-022-01840-5

[2] N. Boumal, *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press, 2023. DOI: 10.1017/9781009166164

[3] M. Jaggi, "Revisiting Frank-Wolfe: Projection-Free Sparse Convex Optimization," in *Proc. 30th Int. Conf. on Machine Learning (ICML)*, vol. 28, pp. 427–435, 2013.

[4] D. Garber and E. Hazan, "Faster Rates for the Frank-Wolfe Method over Strongly-Convex Sets," *Proceedings of the 32nd International Conference on Machine Learning*, pp. 541–549, 2015. URL: https://proceedings.mlr.press/v37/garbera15.html

[5] M. D. Canon and C. D. Cullum, "A Tight Upper Bound on the Rate of Convergence of the Frank-Wolfe Algorithm," *SIAM Journal on Control*, vol. 6, no. 4, pp. 509–516, 1968. DOI: 10.1137/0306032

[6] R. Tibshirani, "Frank-Wolfe Method," Lecture notes, Convex Optimization 10-725, Carnegie Mellon University, 2018. Available: https://www.stat.cmu.edu/~ryantibs/convexopt-F18/lectures/frank-wolfe.pdf

[7] S. Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 2023. DOI: 10.48550/arXiv.2311.05313

[8] A. S. Nemirovski and M. J. Todd, "Interior-Point Methods for Optimization," *Acta Numerica*, vol. 17, pp. 191–234, 2008. DOI: 10.1017/S0962492906370018

[9] R. Bhatia, *Positive Definite Matrices*. Princeton University Press, 2007.

[10] F. Nielsen and R. Bhatia, Eds., *Matrix Information Geometry*. Springer, 2013. DOI: 10.1007/978-3-642-30232-9

[11] Wikipedia contributors, "Hermitian Matrix," Wikipedia, The Free Encyclopedia. Available: https://en.wikipedia.org/wiki/Hermitian_matrix

[12] X. Hua, Y. Cheng, H. Wang, Y. Qin, and Y. Li, "Geometric Means and Medians with Applications to Target Detection," *IET Signal Processing*, vol. 11, no. 6, pp. 711–720, 2017. DOI: 10.1049/iet-spr.2016.0547

[13] R. L. Burden, J. D. Faires, and A. M. Burden, *Numerical Analysis*, 10th ed. Cengage Learning, 2016. (Chapter 2.2)

[14] S. N. Chanda, T. DeGroot, and J. Beauchain, "Geodesic Regression," Unpublished manuscript, Section 1, 2025.

[15] S. Chanda, "Notes on Second Order RGD," Unpublished notes, 2025.

[16] A. Alaee Khangha, "Linear Optimization, Lecture 4," Class notes, Math 200, [Your Institution], 2025.

5 Appendix

5.1 PCA Simulation Code

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA

def f(U, S):
  return -np.sum(U.T @ S @ U * np.eye(U.shape[1]))

def grad_f(U, S):
  G = -2 * S @ U
  UtG = U.T @ G
  symUtG = (UtG + UtG.T) / 2
  return G - U @ symUtG

def qr_retraction(Y):
  Q, R = np.linalg.qr(Y)
  d = np.sign(np.diag(R))
  d[d == 0]= 1
  Q = Q @ np.diag(d)
  return Q
```

```
def pca_stiefel(X, k, max_iters=5000, tau = .5, r = 10e-3, alpha = .2):
  n = X.shape[0]
  Xc = X - X.mean(axis = 1, keepdims=True)
  S = Xc @ Xc.T
  U = np.linalg.qr(np.random.randn(n, k))[0]

  for _ in range(max_iters):
    G = grad_f(U, S)
    if np.sum(G*G) < 1e-10:
      break

    f_curr = f(U,S)
    alpha_k = alpha

    while True:
      Y = U - alpha_k * G
      U_new = qr_retraction(Y)
      f_new = f(U_new, S)

      if f_new <= f_curr - r * alpha_k * np.sum(G*G):
        break

      alpha_k *= tau
      if alpha_k < 1e-10:
        U_new = U
        break

    U = U_new

  return U
```

```
data = load_iris()
X = data.data.T
k = 2
```

```
U_riem = pca_stiefel(X, k)

pca = PCA(n_components=k)
pca.fit(X.T)
U_sklearn = pca.components_.T

P_riem = U_riem @ U_riem.T
P_sklearn = U_sklearn @ U_sklearn.T
print("Subspace error:", np.linalg.norm(P_riem - P_sklearn, 'fro'))
```

5.2 HPD Simulation Code

```
    def gradient_riemannian_mean(X_k, As, ws):

  #need to calculate  X 1 /2 log(X1/2 A 1X1 /2)X1/2

  G = np.zeros_like(X_k)
  lam, B = eigh(X_k)
  X_half = B @ np.diag(np.sqrt(np.maximum(lam, 1e-12))) @ B.T
  X_neg_half = B @ np.diag(1 / np.sqrt(np.maximum(lam, 1e-12))) @ B.T

  for Ai, wi in zip(As, ws):
    A_inv = np.linalg.inv(Ai)
    M = X_half @ A_inv @ X_half
    lam_M, BM = eigh(M)
    lam_M = np.maximum(lam_M, 1e-12)
    log_M = BM @ np.diag(np.log(lam_M)) @ BM.T

    G += wi * X_neg_half @ log_M @ X_half
  return G

def riemannian_linear_oracle(G, L, U):
  lam, Q = eigh(G)

  L_hat = Q.T @ L @ Q
  U_hat = Q.T @ U @ Q

  M = .5 * (U_hat - L_hat + (U_hat - L_hat).T)
  P = cholesky(M)

  sign = np.sign(lam)
  diag_sign = np.maximum(-sign, 0.0)
  D = np.diag(diag_sign)

  Z_hat = P.T @ D @ P + U_hat
  Z = Q @ Z_hat @ Q.T

  return Z

def geo_mean(X, Z, alpha):
  lam, B = eigh(X)
  X_half = B @ np.diag(np.sqrt(lam)) @ B.T
  X_neg_half = B @ np.diag(1 / np.sqrt(lam)) @ B.T

  M = X_neg_half @ Z @ X_neg_half
  lam_M, B_M = eigh(M)
```

```
  lam_M = np.maximum(lam_M, 1e-12)
  M_alpha = B_M @ np.diag(lam_M ** alpha) @ B_M.T

  return X_half @ M_alpha @ X_half

def compute_bounds(As, ws):
  L = np.zeros_like(As[0])
  U = np.zeros_like(As[0])

  for Ai, wi in zip(As, ws):
    L += wi * np.linalg.inv(Ai)
    U += wi * Ai

  L = np.linalg.inv(L)

  return L, U

def riemannian_distance(X,Y):
  lam, B = eigh(X)
  lam = np.maximum(lam, 1e-12)
  X_neg_half = B @ np.diag(1 / np.sqrt(lam)) @ B.T

  M = X_neg_half @ Y @ X_neg_half
  lam_M, _ = eigh(M)
  lam_M = np.maximum(lam_M, 1e-12)

  return np.sqrt(np.sum(np.log(lam_M) ** 2))
```

```
    def riemannian_frank_wolfe_geometric_mean(As, ws, X0, max_iters=5000, X_true =
        None):
  X_k = X0
  errors = []

  L, U = compute_bounds(As, ws)

  for k in range(max_iters):
    if X_true is not None:
      errors.append(riemannian_distance(X_k, X_true))

    G = gradient_riemannian_mean(X_k, As, ws)
    Z_k = riemannian_linear_oracle(G, L, U)
    alpha_k = 2 / (k + 2)
    X_k = geo_mean(X_k, Z_k, alpha_k)
  X_bar = X_k

  return X_bar, errors
```

```
    def matrix_geometric_mean_reference(As, ws, max_iters=200):
    """Gradient descent on the Riemannian manifold as reference solution."""
    X = As[0].copy()
    for _ in range(max_iters):
        G = gradient_riemannian_mean(X, As, ws)
        lam, B = eigh(X)
        X_half = B @ np.diag(np.sqrt(np.maximum(lam, 1e-12))) @ B.T
        X = geo_mean(X, X - 0.01 * X_half @ G @ X_half, 1.0)  # small step
    return X
```